Midterm Exam

● Graded

Student

KAI-HUNG) 蕭凱鴻 (HSIAO

Total Points

87 / 100 pts

Question 1

(no title) **10** / 10 pts

✔ **– 0 pts** Correct

– 1 pt No/Wrong time complexity analysis

– 1 pt No/Wrong extra space complexity analysis

– 2 pts No/Wrong correctness explanation

– 2 pts Minor mistake

– 4 pts Two minor mistakes

– 6 pts Major mistake

– 10 pts Blank/Wrong answer

+ 1 pt Reasonable Efforts

Question 2

(no title) **10** / 10 pts

✔ **– 0 pts** Correct

– 1 pt Very minor mistake

– 2 pts Minor mistake

– 4 pts Two minor mistakes

– 6 pts Major mistake

– 10 pts Blank / Wrong answer

+ 1 pt Reasonable efforts

+ 2 pts Very reasonable efforts

2 Great job!

Question 3

(no title) **10** / 10 pts

✔ **– 0 pts** Correct

– 1 pt No / wrong time complexity explanation

– 1 pt No / wrong extra space complexity explanation

– 2 pts No / wrong correctness explanation

– 2 pts Minor mistake

– 4 pts Two minor mistakes

– 6 pts Major mistake

– 10 pts Blank / wrong answer

+ 1 pt Reasonable efforts

– 1 pt Pseudo code exceeds 30 lines

Question 4

(no title) **10** / 10 pts

✔ **– 0 pts** correct

– 2 pts minor mistake

– 4 pts two minor mistakes

– 5 pts wrong mathematical induction

– 6 pts major mistake (e.g. only write down n = 1 situation)

– 10 pts wrong / blank answer

+ 1 pt reasonable efforts

Question 5

(no title) **10** / 10 pts

✔ **– 0 pts** Correct

– 1 pt No/Wrong time complexity analysis

– 1 pt No/Wrong extra-space complexity analysis

– 2 pts No/Wrong correctness explanation

– 2 pts One minor mistake

– 4 pts Two or more minor mistakes

– 6 pts Major mistake

– 10 pts Blank/Wrong answer

+ 1 pt Reasonable efforts

Question 6

(no title) **10** / 10 pts

✔ **– 0 pts** Correct

– 1 pt No/Wrong time complexity analysis

– 1 pt No/Wrong extra-space complexity analysis

– 2 pts No/Wrong correctness explanation

– 2 pts Minor mistake

– 4 pts Two minor mistake

– 6 pts Major mistake

– 10 pts Blank/Wrong answer

+ 1 pt Reasonable Efforts

Question 7

(no title) **Resolved** **8** / 10 pts

– 0 pts Correct

– 1 pt No/Wrong time complexity analysis

– 1 pt No/Wrong extra-space complexity analysis

– 2 pts No/Wrong correctness explanation

✔ **– 2 pts** Minor mistake

– 4 pts Two minor mistake

– 6 pts Major mistake

– 10 pts Blank/Wrong answer

+ 1 pt Reasonable Efforts

3 +1

4 +1

Regrade Request **Submitted on: Apr 24**

這題我錯的地方是在 ans=0 那邊，應該是 ans=1 答案才會是對的。請問這樣的一個小小小 typo 直接 -2 會不會有點太狠了 QQ

You can't argue with criteria.

Reviewed on: Apr 24

Question 8

(no title) **Resolved** **5** / 10 pts

- **– 0 pts** Correct
- **– 2 pts** Minor Mistake
- **– 4 pts** Two Minor Mistakes
- ✔ **– 5 pts** No/Wrong explanation for why each while loop runs at most O(k) times
- **– 5 pts** No/Wrong explanation for why each element is moved at most O(k) times
- **– 6 pts** Major Mistake
- **– 10 pts** Blank/Wrong Answer
- **+ 1 pt** Reasonable Efforts

Regrade Request **Submitted on: Apr 28**

助教好，這題我寫的可能有點粗略，但我要表達的事情應該是正確的沒錯，於是再這裡重複一次，希望可以拿到多一點分數，我自己私心覺得應該可以滿分。
我的 claim 是每次 while 迴圈只會執行最多 k 次就會跳出去了。
我的 proof 是假如有哪次 while 迴圈執行了超過 k 次，那就會和題敘的所有偏移量不超過 k 矛盾，因為這代表存在某個 key 在做題目程式碼第七行 a[i+1] 的時候往 index 小的方向移動了超過 k 步。就算不是當前的 key 發生的，為了讓這個 key 滿足偏移不超過 k 的性質，將來也必須會有某個值往前移更多才能把當前的 key 往 index 大的地方擠回去，但屆時又會違反偏移量不超過 k 的性質。
再麻煩助教重新給一下分數，謝謝，辛苦了！

同學好，你的作答中，proof的第一個j（key從i到j）是該元素insert時的暫時位置，第二個j（與|i-j|<=k矛盾）則是元素的最終位置，因此這個證明是有問題的。
必須加上regrade request中「就算不是當前的 key 發生的，為了讓這個 key 滿足偏移不超過 k 的性質，將來也必須會有某個值往前移更多才能把當前的 key 往 index 大的地方擠回去，但屆時又會違反偏移量不超過 k 的性質。」這段，才比較能說明為何可以將k-sorted的限制條件套用在insert的位置上。然而作答內容中並不包含以上說明，因此這題會維持原本的批改結果。

Reviewed on: Apr 28

Regrade Request **Submitted on: Apr 28**

可是少了那個敘述，不應該被歸在 minor mistake 嗎？
我發現我只是少寫出，可以直接考慮向 index 小的方向偏移最多的人。處理完這個人後，不會有任何人可以把他往 index 大的方向擠，我寫的就是對的了。
我覺得少寫出這個 point 就直接被歸在 no/wrong explanation 有點奇怪，謝謝助教。

同學好，你的proof中兩次用到"j"的意思是不同的，所以這邊不能直接使用|i-j|<=k說明矛盾。扣除這句後，剩餘部分無法說明為何key向左移動超過k步會無法滿足k-sorted，因此這題的分數是給wrong explanation。
如果考卷有寫到類似你regrade request中的說明，但中間有小瑕疵，這樣才會給minor mistake。希望有幫助你更了解批改標準！

Reviewed on: Apr 29

Question 9

(no title) **10** / 10 pts

✔ **– 0 pts** Correct

– 2 pts Minor mistake

– 4 pts Two minor mistakes

– 2 pts Missing or not specifying n_0

– 2 pts Missing or not specifying c

– 6 pts Major mistake/More than two minor mistakes

– 10 pts Blank/Wrong Answer

+ 1 pt Reasonable Efforts/Disprove with reasonable effort

Question 10

(no title) **4** / 10 pts

If chose to prove

– 0 pts Correct

– 2 pts Minor Mistake

– 3 pts Proof relies on continuity / differentiability

– 3 pts Proof fails for f(n) or g(n) $\rightarrow$0 or g(n) = 0

– 4 pts ≥ 2 Minor mistakes

✔ **– 6 pts** Major Mistake

– 9 pts Reasonable Efforts

If chose to disprove

– 0 pts Click here to replace this description.

– 2 pts Minor Mistake

– 4 pts ≥ 2 Minor Mistakes

– 6 pts Major Mistake

– 9 pts Reasonable Efforts

– 10 pts Empty / Irrelevant

1 Please review logirithm

SCHOOL ID: B13902046 **NAME:** 陳鈺鴻

Midterm Examination Problem/Answer Sheet

TIME: 04/08/2025, 13:20–16:10

This is an open-book exam. You can use any printed materials as your reference during the exam. Any electronic devices are not allowed. Both English and Chinese (if suited) are allowed for answering the questions. We do not accept any other languages.

Any form of cheating, lying or plagiarism will not be tolerated. Students can get zero scores and/or get negative scores and/or fail the class and/or be kicked out of school and/or receive other punishments for those kinds of misconducts.

There are 10 problems. 5 of them are marked with * and are supposedly simple(r); 5 of them are marked with ** and are supposedly hard(er). Each problem is worth 10 points. **We will calculate your midterm exam score by re-weighting your best two problems by 150%—scaling them to 15 points each, and your worst two problems by 50%—scaling them to 5 points each.** For each problem, you can answer it with one and only one side, as the problem/answer sheet indicates. For the odd-numbered problems, please write down your SCHOOL ID (top-left) and your NAME (top-right) on the sheet first. You should keep your answers as *concise* as possible to facilitate grading.

1. (*) **Write down your SCHOOL ID and NAME above, and your SOLUTION below.**

   A *valley* array is a size-n array a that contains elements that are first strictly decreasing: $a[1] > a[2] > a[3] > \cdots > a[v]$, and then strictly increasing $a[v] < a[v+1] < \cdots < a[n]$. There is no guarantee on whether there are duplicates within the array other than the orders above. When knowing a valley array a along with n, design an $O(\log n)$-time and $O(1)$-extra-space algorithm to find and return v, the index of the "valley." You can assume that there exists a unique v with $1 < v < n$. Write down the pseudo code for your solution within 30 lines. Briefly explain why the pseudo code is correct under the required time and extra-space complexity.

```
function valley(n, a[])
    l = 2, r = n-1
    while l < r :
        m = ⌊(l+r)/2⌋
        if a[m] < a[m+1]
            r = m
        else
            l = m+1
    return l
```

Idea: binary search for the first index id such that $a[id] < a[id+1]$

During the progress, make sure that the potential "id" be in interval $[l, r]$.

Complexity: time $O(\log n)$

since the interval $(l, r]$ shorten by about half every iteration, becoming lengthed 1 after about $\log_2 n$ time

Space: only variables l, r, m.

$O(1)$-extra-space.

2. (**) **Write down your SOLUTION below.**

Consider an array b of length n that contains distinct integers. Consider another array a of length m that contains a subset of distinct integers in b. The next-larger element of $a[i]$ in b is defined as $b[j]$ with the smallest $j > i$ such that $b[j] > a[i]$. If no such element exists in b, the next-larger element is NIL. For instance, if

$$a = [8, 9, 6, 4], b = [4, 7, 9, 8, 6]$$

the array of its next-larger elements are

$$[9, \text{NIL}, 8, 6].$$

Design an $O(n + m \log n)$-time and $O(n)$-extra-space algorithm that *modifies* a to the array of its next-larger elements in b. Consider either a 1-based or 0-based array is fine as long as your convention is clear from the context.

→ a, b, c are all 0-based

Yes, this is (super-)upgraded from the 2023 Midterm exam problem. ;-)

```
function Get-nearest-right-larger(m, n, a[], b[]):
    c[] ← an array can accomodate (m+1) elements
    c[0] = inf
    cnt = 1.
    for i = m-1 to 0.
        if i < n
            l = 0, r = cnt
            while l+1 < r
                m = ⌊(l+r)/2⌋
                if c[m] ≤ a[i].
                    r = m.
                else l = m
            a[i] = c[l].
            if a[i] = inf : a[i] = nil.
        while c[cnt-1] < b[i] : cnt = cnt-1
        c[cnt] = b[i], cnt = cnt+1.
    return a[].
```

SCHOOL ID: B13902046 **NAME:** 蕭詠鴻

3. (*) **Write down your SCHOOL ID and NAME above, and your SOLUTION below.**

The key design of the QUICKSORT algorithm on an array depends on the fact that one can partition any size-n array a with respect to some pivot point p to a pile containing $a[i] \leq p$ and another pile containing $a[j] > p$ in $O(n)$ time. Now, when given a size-n singly linked list ℓ represented by $\ell.head$ with $n > 0$, and a separate pivot node p, design an algorithm to divide ℓ into two singly linked lists, one containing all nodes with $node.key \leq p.key$ (not including p itself), and the other containing all nodes with $node.key > p.key$, in $O(n)$ time and $O(1)$ of extra space. The nodes in the two singly linked lists can be put in any order of your choice. Return the heads to the two singly linked lists. You can assume access to not just $\ell.head$ but also $\ell.tail$ if needed. Write down the pseudo code for your solution within 30 lines. Briefly explain why the pseudo code is correct under the required time and extra-space complexity.

```
function split(l, p)
    ls, lb ← linked list for smaller and bigger elements respectively
    ls.head = Nil , lb.head = Nil.
    cur = l.head
    while cur ≠ Nil
        tmp = cur.next.
        if cur.key ≤ p.key :
            cur.next = ls.head ; ls.head = cur
        else :
            cur.next = lb.head ; lb.head = cur.
        cur = tmp.
    return ls.head, lb.head.
```

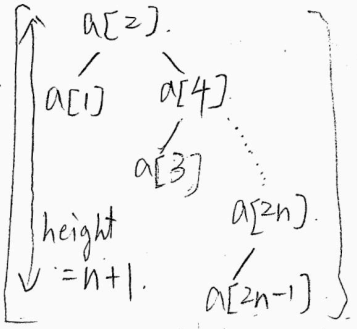
For bigger and smaller lists, maintain the last nodes added to them
Iterate the original list, check which list it should be inserted to
then update the new list's head.
time complexity : O(n) for traversal
space complexity : O(1) - extra space, only variables ls.head
lb.head, cur, tmp

4. (**) **Write down your SOLUTION below.**

Consider a length $2n$ sequence of keys $a[1] < a[2] < \cdots < a[2n]$. Prove by mathematical induction that inserting the keys into a binary search tree with order

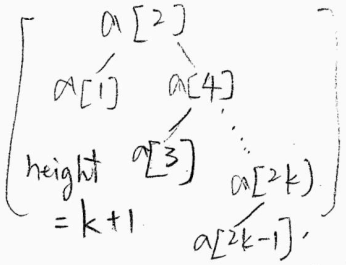
$$a[2], a[1], a[4], a[3], \ldots, a[2n], a[2n-1]$$

results in a binary search tree with height $n+1$. Note that a root-only tree is of height 1. The rigor of your proof will be considered during grading.

Claim: 樹的樣子會是 

for all n.

當 n=1 時，樹是 $\begin{bmatrix} a[2] \\ a[1] \end{bmatrix}$，命題成立.

假設 n=k 時命題成立，樹會是 

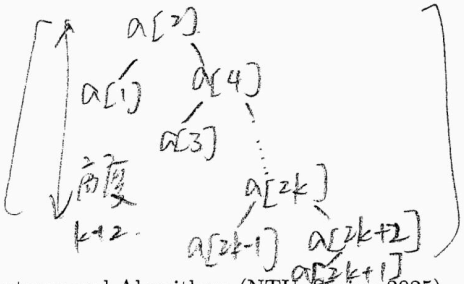
，則當 n=k+1 時會延用 n=k 的

插入 a[2k+2]，由於 a[2k+2] 比所有 a[2], a[4] ... a[2k] 都

會放在 a[2k] 的右子樹，高度此時與 n=k 時相同

插入 a[2k+1]，a[2k+1] 比 a[2], a[4] ... a[2k] 大，但比 a[2

會放在 a[2k+2] 的左子樹，高度增加1變為 (k+1)+1

新的樹： 

命題亦成立.

故由數學歸納法得證

n=N 時，樹高為 N+1

對所有正整數都成立。

SCHOOL ID: B13902046 **NAME:** 蔡劼鴻

5. (*) **Write down your SCHOOL ID and NAME above, and your SOLUTION below.**

Consider a queue implemented with a circular array of a known size n, where the queue is initially empty. Given a sequence of m `enqueue(key)` and `dequeue()` operations, design an $O(m)$-time and $O(1)$-extra-space algorithm that computes whether the queue overflows (i.e. exceeds its capacity n) or underflows (i.e. when `dequeue()` is called on an empty array) at any of the operations *without actually implementing the queue*. If there is an overflow or an underflow, return `OVERFLOW` or `UNDERFLOW`, depending on which happens first; if not, return `SAFE`. Write down the pseudo code for your solution within 30 lines. Briefly explain why the pseudo code is correct under the required time and extra-space complexity.

```
function check(n, m, ops[])
    sz = 0
    for i = 1 to m
        if ops[i] = enqueue(key)
            sz = sz + 1
        else sz = sz - 1
        if sz = -1
            return UNDERFLOW
            exit function
        else if sz = n+1
            return OVERFLOW
            exit function
    return SAFE
```

During the progress, maintain the current size of the queue. Since the size vary by 1 each operation, we can check if $sz = n+1$ or $sz = -1$ for OVERFLOW and UNDERFLOW respectively.

Only 1 traversal, so $O(m)$ time

Only 1 variable sz., $O(1)$-extra-space.

6. (**) **Write down your SOLUTION below.**

Given n integers in an array with $n > 2k$. Assume that we want to calculate a more robust estimate of the mean by considering the average of the integers without the smallest k integers and the largest k integers, with time complexity $O(n \log k)$ and extra-space complexity $O(k)$. Design an algorithm to return the more robust estimate. You can directly use any data structures and their associated algorithms that are taught in class if needed. Write down the pseudo code for your algorithm within 30 lines. Briefly explain why the algorithm works correctly under the time and space complexity. Consider either a 1-based or 0-based array is fine as long as your convention is clear from the context.

No, this is NOT EXACTLY the same as your HW2 problem. ;-), as the time and extra-space complexities are changed.

```
function Robust-Estimate( n, k, arr[] ).
    H_m ← An empty min-heap.  )→ contain at most
    H_M ← An empty max-heap       k elements.
sum=0.) for all value in arr[]:
        Insert( H_m, value)
        Insert( H_M, value).
        if size(H_m) = k+1 : Pop(H_m).
        if size(H_M) = k+1 : Pop(H_M)
        sum = sum + value.
    while not Empty(H_m) : sum = sum - Pop(H_m).
    while not Empty(H_M) : sum = sum - Pop(H_M)
    return sum/(n-2k).
```

The min-heap is used in order to keep the largest elements checked, but we only care about k largest at most, Similar for max-heap. If the heaps' size are larger than k, remove the extra one element we don't care, keeping the heap size smaller or equal to k. Each insertion take $O(\log k)$ time, n insertion means $O(n \log k)$ in total. 2 heaps cost $O(k)$ extra space.

SCHOOL ID: B13902046 **NAME:** 蕭凱鴻

7. (*) **Write down your SCHOOL ID and NAME above, and your SOLUTION below.**

Consider a size-n binary tree with a given n, where each node has a field *count* that stores the number of descendants of the node, where a leaf node has $count = 0$ and the root node has $count = n - 1$. You can assume that each node the binary tree has the usual *left* and *right* fields, as well as the *parent* one (if needed). Nevertheless, for one and only one of the nodes within the binary tree, the value of *count* is wrongly stored. Design a $O(n)$ time and $O(h)$-extra-space algorithm that identifies and returns the node that contains the wrong *count*, where h is the height of the tree. Write down the pseudo code for your algorithm within 30 lines. Briefly explain why the algorithm works correctly under the time and extra space complexity.

```
function DFS (cur)
    ret = Nil , ans = 0.
    if cur.left ≠ Nil
        tmp = DFS(cur.left)
        if tmp ≠ Nil : return tmp.
        ans = ans + cur.left.count
    if cur.right ≠ Nil
        tmp = DFS (cur.right)
        if tmp ≠ Nil : return tmp
        ans = ans + cur.right.count
    if cur.count ≠ ans.
        return cur
    else : return Nil
```

During DFS progress, if one of the children contain the only answer, just return it. Or the descendents' counts are all correct, check for if itself's count is wrong.

Recursion need $O(n)$ time traversing all n nodes and $O(h)$ space since h is also the largest recursion deep.

(each single recursion use $O(1)$ variables.)

8. (**) **Write down your SOLUTION below.**

Consider the following INSERTION-SORT algorithm.

INSERTION-SORT(arr)

```
1  for m = 2 to a.length
2      key = a[m]
3      i = m − 1
4      while i > 0 and a[i] > key
5          a[i + 1] = a[i]
6          i = i − 1
7      a[i + 1] = key
```

Consider n numbers $a[1], a[2], \ldots, a[n]$ that are distinct. Assume that the array a is k-sorted for some constant $k \ll n$. That is, when sorting the array a to some b, where each $a[i]$ is moved to $b[j]$, $|i - j| \leq k$. Prove that the INSERTION-SORT algorithm above runs within $O(nk)$ time. That is, it is adaptive.

上列 pseudo-code 的第 4~6 行會讓

a[i] = b[j] 且 i < j 的人往右動 1 步

claim：每次最多只有 k 個人，即 while 每次

進去時跑最多 k 次就會跳出 while

proof：若有大於 k 個人往右動一步，代表

讓此時的 key 從 i 到 j (向左) 至少

移了 (k+1) 個，與 |i-j| ≤ k 矛盾

每次跑 while 的時間是 O(k) 的

最多 n 次會執行第一個 while，故總時間

O(nk).

SCHOOL ID: B13902046 **NAME:** 蕭詠鴻

9. (*) **Write down your SCHOOL ID and NAME above, and your SOLUTION below.**

For the next two problems, please prove the results using the following definition of the asymptotic notations (which is exactly the same as the notation you learned in our class).

> Suppose that f and g are non-negative functions defined on $\mathbb{N}$ (if you need, you can also assume them to be defined on $\mathbb{R}$). We call $f(n) = O(g(n))$ if and only if there exist positive numbers $c \in \mathbb{R}^+$ and $n_0 \in \mathbb{N}$ such that
>
> $$f(n) \leq cg(n) \text{ for all } n \geq n_0.$$
>
> In addition, $f(n) = O(g(n))$ if and only if $g(n) = \Omega(f(n))$.

For your proof, you can use the following. Please state where those results are from (to your belief) with a line or two if you choose to use them.

- any results that have been proved in our class—please be extra careful on this. Our experience is that students sometimes "think" some results have been proved in the class, but actually, those results have not been proved. So it is very important to understand what have been proved and what have not been proved. **If you are not sure, the best strategy is to write down the proof by yourself!**
- any results in related sections of your textbook, regardless of whether they have been taught.
- any results in your high-school Math class and your fundamental Calculus or Discrete Math classes.

Prove or disprove that for any $p \in [0, 1)$, $n! = \Omega(n^{pn})$. If needed, you can use Stirling's approximation $\lim_{n\to\infty} \frac{n!}{\sqrt{2\pi n}(\frac{n}{e})^n} = 1$ without proving.

Yes, this is EXACTLY the same as your HW1 problem. Please note that copying from the solution sketch will likely not get you all points as the sketch was deliberately made to be overly simple. ;-)

$$\lim_{n\to\infty} \frac{n!}{n^{pn}} = \lim_{n\to\infty} \frac{n!}{\sqrt{2\pi n}(\frac{n}{e})^n} \cdot \frac{\sqrt{2\pi n}(\frac{n}{e})^n}{n^{pn}}$$

$$= 1 \cdot \lim_{n\to\infty} \sqrt{2\pi n} \cdot \left(\frac{n^{1-p}}{e}\right)^n = \infty$$

[Since $p \in [0,1)$, $1-p > 0$]

By definition, there's some n_0 for every $N > 0$ such that $n \geq n_0$ implies $\frac{n!}{n^{pn}} \geq N$, $n! \geq N \cdot n^{pn}$

Take any positive C, there's some n_0 s.t. for all $n \geq n_0$, $n! \geq C \cdot n^{pn}$.

Hence, $n! = \Omega(n^{pn})$ for any $p \in [0, 1)$.

10. (**) **Write down your SOLUTION below.**

For non-negative functions $f(n)$ and $g(n)$ Assume that $\ln(1+f(n)) = O(\ln(1+g(n)))$. Prove or disprove that $f(n) = O(g(n)^k)$ for some $k > 0$.

$\ln(1+f(n)) = O(\ln(1+g(n)))$.

$\Rightarrow$ there's some (c, n_0) s.t. $n \geq n_0 \rightarrow \ln(1+f(n)) \leq c \ln(1+g(n))$

$1+f(n) \leq e^c(1+g(n))$ 1

Assume that $f(n) = O(g(n)^k)$, which means there's some c' s.t. $f(n) \leq c'[g(n)]^k$ for large n.

$c' \geq \frac{f(n)}{g(n)^k}$

$\frac{(1+g(n))e^c}{g(n)^k} \geq \sqrt[2]{g(n)}\, e^c$.